

Vizsgaremek pontozás részletezése

1. A szoftver átfogó értékelése

5 pont

- **Ötlet és életszerűség:** A projekt egy valós, nem triviális problémát céloz meg, nem csupán egy tantermi feladatot old meg. 1 pont
- **Komplexitás:** A szoftver több, egymással logikusan összekapcsolódó modult vagy fő funkciót tartalmaz (pl. felhasználókezelés, adatvizualizáció, foglalási rendszer stb.). 1 pont
- **Felhasználói élmény (UX):** A felület logikusan felépített, a felhasználó különösebb segítség nélkül is képes navigálni és használni a fő funkciókat. 1 pont
- **Kidolgozottság:** A szoftver kész termék benyomását kelti. Nincsenek szembevető hibák, a dizájn egységes, a kinézethez használt css keretrendszer elemei helyesen, rezponzívan és esztétikusan vannak felhasználva. 2 pont

2. Adatbázis-tervezés és megvalósítás

5 pont

- **Adatbázis séma tervezése:** A séma megfelel a 3. normálformának (3NF), elkerüli az adatredundanciát. Az adattípusok és méretei helyesen vannak megválasztva (pl. VARCHAR vs. TEXT, INT vs. DECIMAL). A kapcsolatok (külső kulcsok, megszorítások) definiálva vannak és tükrözik a valós logikát. 2 pont
- **ORM használat (Sequelize):** A modellek helyesen vannak definiálva a Sequelize-ben. Az asszociációk (hasOne, hasMany, belongsToMany) szakszerűen vannak implementálva a modellek között. 1 pont
- **Teljesítmény és adatintegritás:** A gyakran keresett oszlopokon vannak csak indexek. Tranzakciókat használnak ott, ahol több, egymástól függő adatbázisművelet történik (pl. egy foglalás rögzítése és a raktárkészlet csökkentése). 2 pont

3. Szerver oldali komponens (backend)

10 pont

- **Architektúra és projektstruktúra (Node.js/Express):** A szerver oldali kód tiszta, logikus mappaszerkezetet követ (pl. routes, controllers, services, models). A felelősségi körök el vannak választva (a route-ok csak a kérést fogadják, a controllerek vezérlik a logikát). 2 pont
- **REST API megvalósítás:** Az API végpontok követik a REST konvenciókat (pl. GET /api/users, POST /api/users). A HTTP metódusok (GET, POST, PUT, DELETE) helyes használata és a megfelelő HTTP státuszkódok (200, 201, 400, 404) visszaküldése. 2 pont
- **Biztonság:** A felhasználói jelszavak hashelve (pl. bcrypt csomaggal) vannak tárolva. A védett végpontok elérése autentikációhoz kötött (pl. JWT token alapú rendszer middleware-rel). Megvalósítottak autorizációt is (pl. admin vs. user szerepkörök). 2 pont
- **Validáció és hibakezelés:** Minden bejövő adatot (request body, params) validálnak a szerver oldalon (pl. express-validator segítségével). Létezik egy központi hibakezelő middleware, ami egységes formátumú hibaüzeneteket küld vissza. 2 pont
- **Külső szolgáltatások integrációja:** Legalább egy külső szolgáltatást (pl. levélküldés nodemailer-rel regisztrációkor vagy jelszó-visszaállításhoz) szakszerűen integráltak. 1 pont
- **API dokumentáció:** Az API végpontok dokumentálva vannak (pl. egy Postman kollekcióval vagy Swagger/OpenAPI generálásával). 1 pont

4. Asztali használatra készült kliens oldali komponens (frontend)

10 pont

Ebben a részben a .NET környezetben (WPF, Windows Forms vagy Avalonia) készített, telepíthető futtatható állományt (.exe) értékeljük.

A vizsgaremek követelményrendszere lehetőséget biztosít arra, hogy a vizsgázó a natív asztali alkalmazás fejlesztését kiváltsa egy kiemelkedő minőségű, minden eszközön (desktop és mobil) tökéletesen működő webes alkalmazással, amit kifordít (pl.: electron) futtatható állománnyá.

Amennyiben a vizsgázó nem készít különálló C# alapú (WPF/Forms) asztali alkalmazást, az értékelés a következő pontszám-tükrözési elv alapján történik: Az asztali kliens pontszáma automatikusan megegyezik a mobil és webes kliens elért pontszámmal. Amennyiben nem fordítja ki futtatható állománnyá 0 pont adható az asztali kliens részre.

Ennek következményei (Dupla kockázat elve): Ez a választás fokozott kockázattal jár. Mivel a webes megoldás értékelése kerül megduplázásra, minden egyes hiba, hiányosság vagy levont pont a webes résznél kétszeres pontlevonást eredményez a végső elszámolásban

Példa: Ha a webes alkalmazás csak 10/8 pontot ér, akkor az asztali alkalmazás rész is ennyit ér, így 20/16 pontot kap.

- **Architektúra és Kódminőség:** A kód követi a választott technológia ajánlásait. WPF: MVVM (Model-View-ViewModel) minta használata elvárt, az üzleti logika elválik a megjelenítéstől. WinForms esetén: Az eseménykezelők (Event Handlers) nem tartalmaznak nehéz üzleti logikát, a kód osztályokra tagolt. 2 pont
- **Adatkötés (Data Binding) és Megjelenítés:** A felületen megjelenő adatok dinamikusan kötődnek a kódhoz. Listák, táblázatok (DataGrid, ListView) használata az adatok strukturált megjelenítésére. A modellek (DTO-k) pontosan tükrözik a backendről érkező JSON struktúrát. 2 pont
- **API Kommunikáció (HttpClient):** A backenddel való kommunikáció aszinkron módon (async/await), a HttpClient (vagy RestSharp) szakszerű használatával történik. A JSON szerializáció/deszerializáció (Newtonsoft vagy System.Text.Json) helyesen működik. A UI nem fagy le az adatlekérés alatt. 2 pont
- **Funkcionalitás és CRUD műveletek:** Az asztali alkalmazásban is megvalósulnak a legfontosabb funkciók (pl. listázás, szűrés, módosítás: létrehozás/szerkesztés/törlés). A funkciók működése stabil. 2 pont
- **Felhasználói élmény és Hibatűrés:** Az alkalmazás kezeli a hálózati hibákat (pl. ha nem elérhető a szerver, nem omlik össze, hanem hibaüzenetet ad). A beviteli mezők validációja megoldott. A felület elrendezése (Layout: Grid, StackPanel, Anchor) alkalmazkodik az ablak átméretezéséhez. 2 pont

5. Mobil használatra készült kliens oldali komponens

10 pont

Ebben a részben a modern frontend keretrendszerekkel készített, böngészőben futó alkalmazást értékeljük. A "mobil komponens" itt a reszponzív webes megvalósítást jelenti.

- **Komponens alapú architektúra (Vue/React/Angular):** Az alkalmazás jól definiált, újrahasznosítható komponensekből épül fel. A szülő-gyerek komponens kommunikáció (props/inputs) tiszta és következetes. 2 pont

- **Állapotkezelés és Routing:** Az alkalmazás állapotát szakszerűen kezelik (pl. Vuex/Pinia, Redux, Context API, Service-ek). A navigációt kliens oldali router (vue-router, react-router, angular router) valósítja meg, biztosítva az oldal újratöltése nélküli nézetváltást és a védett útvonalak (Guards) kezelését. 2 pont
- **API kommunikáció és aszinkron műveletek:** Az API hívásokat (axios vagy fetch segítségével) egy elkülönített réteg (pl. service) kezeli. Az aszinkron folyamatok (adatbetöltés, mentés) alatt a felhasználó visszajelzést kap (pl. betöltésjelző, hibaüzenet). A hibakezelés (pl. 401, 404, 500) megoldott. 2 pont
- **Reszponzivitás és Mobil UX:** A felület "Mobile First" szemlélettel vagy rezponzív módon készült. Egy tetszőleges CSS keretrendszer (pl. Bootstrap, Tailwind) grid rendszerét és a media query-eket helyesen használják. Mobilon a menü (pl. hamburger), a gombok mérete és az elrendezés érintésbarát, nincs oldalirányú görgetés. 2 pont
- **UI/UX és Design:** Az alkalmazás vizuálisan egységes, modern megjelenésű. A formok validációja (pl. kötelező mezők, email formátum) felhasználóbarát módon, valós időben történik a felületen. A képek optimalizálva vannak, és nincsenek feleslegesen nagy, a betöltést lassító erőforrások. Minden asztali funkció elérhető és használható a mobil felületen is, kompromisszumok nélkül. 2 pont

6. A kód minősége 3 pont

- **Olvashatóság és konvenciók:** A változók, függvények, komponensek elnevezése beszédes és követi az adott nyelv/keretrendszer konvencióit (pl. camelCase, PascalCase). 1 pont
- **Struktúra és DRY elv:** A kód nincs ismétlésektől terhelve (Don't Repeat Yourself). A függvények rövidek és egyetlen feladatot látnak el (Single Responsibility Principle). 1 pont
- **Karbantartathatóság:** A kód könnyen érthető egy külső fejlesztő számára is. A fontosabb részeknél kommentek vannak és a "miért"-et magyarázzák, nem a "mit". 1 pont

7. A dokumentáció minősége és részletezettsége 2 pont

- **Telepítési útmutató:** A dokumentáció tartalmaz egy "getting started" szekciót, ami leírja a projekt klónozását, a függőségek telepítését (npm install), és az indításhoz szükséges lépéseket (pl. .env fájl létrehozása, adatbázis beállítása). Tartalmaz egy Ho-To Guides részt a főbb funkciók használatáról 1 pont
- **Rendszerleírás:** Tartalmaz egy rövid leírást a szoftver céljáról, a főbb funkciókról és a felhasznált technológiákról. Tartalmaz egy részletes leírást vagy egy egyszerűsített ábra a rendszer felépítéséről. 1 pont

8. A szoftver tesztelésének bemutatása 3 pont

- **Unit tesztek megléte:** A csapat bizonyítja, hogy írtak unit teszteket a kódjuk minőségbiztosítására. A forráskódban léteznek a tesztesetek a kritikus logikai részekre (pl. backend service-ek, frontend segédfüggvények). 2 pont
- **Unit tesztek minősége:** A bemutatott tesztek nem csak triviális eseteket fednek le, hanem vizsgálnak peremeseteket vagy várható hibás működést is (pl. hibás bemeneti adatra a függvény a várt hibát dobja-e). 1 pont

9. A szoftver bemutatása és angol kommunikáció 3 pont

- **Prezentáció felépítése:** A bemutató logikusan felépített (cél, demo, technikai részletek, tanulságok), a diák magabiztosan, érthetően beszél 1 pont
- **Szakmai mélység:** A vizsgázó képes megindokolni a technológiai döntéseit ("Miért React és nem Vue?", "Miért JWT autentikáció?"), és érti a saját kódjának működését. 1 pont

- **Angol nyelvi kommunikáció:** A vizsgázó egy szakmai kérdésre angolul, helyes terminológiát használva, összefüggően tud válaszolni. 1 pont

10. A csapatmunka megvalósítása

4 pont

- **Verziókezelési (Git):** A Git history-ből egyértelműen látszik a feature branch használata. A munkát külön ágakon végezték, és Pull Request-eken (vagy Merge Request) keresztül olvasztották be a fő ágba. 2 pont
- **Hozzájárulások és minőség:** A commit log-ból látszik, hogy minden csapattag arányosan hozzájárult a projekthez. A commit üzenetek informatívak, követnek valamilyen konvenciót (pl. leírják, hogy mit és miért módosítottak). 2 pont